

Prompt:

What is the steps to 'Implement Playwright with TypeScript'? share the steps, after my permissions implement the changes.

Asking which level of implementation to perform next so changes can be applied correctly — choose Full to add packages, configs, example tests, scripts, and CI workflow; choose Minimal to add only packages and configs; or choose to keep this as a guidance-only response.

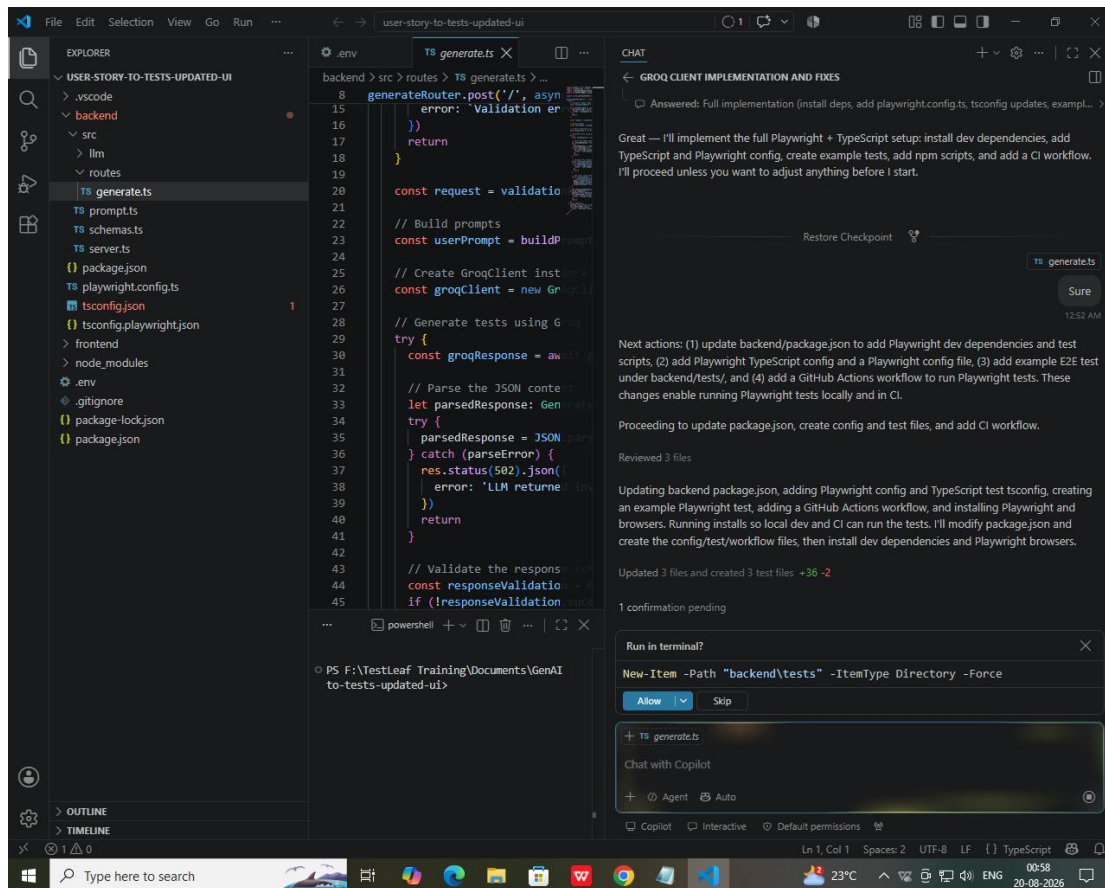
Test

The screenshot shows a Visual Studio Code editor with a project named 'user-story-to-tests-updated-ui'. The Explorer sidebar on the left shows the file structure, including 'backend', 'src', 'routes', and 'generate.ts'. The main editor area displays the content of 'generate.ts', which is a TypeScript file for a REST API endpoint. The code includes a POST route for '/generate' that uses the 'groq' library to generate text based on a prompt. The code is as follows:

```
backend > src > routes > TS generate.ts > ...
8 generateRouter.post('/', async (req, res) => {
15   // Generate a prompt based on the request body
16   const prompt = req.body.prompt;
17   // Generate a response using the Groq client
18   const response = await generateText(prompt);
19   // Return the response as JSON
20   res.json(response);
21 })
22 // Build prompts
23 const userPrompt = buildPrompt(prompt);
24 // Create GroqClient instance
25 const groqClient = new GroqClient({
26   apiKey: process.env.GROQ_API_KEY,
27 });
28 // Generate tests using Groq
29 try {
30   const groqResponse = await groqClient.generateText(
31     userPrompt,
32     {
33       // Parse the JSON content
34       let parsedResponse: GeneratedText = JSON.parse(
35         response.text);
36     } catch (parseError) {
37       res.status(502).json({
38         error: 'LLM returned invalid JSON'
39       });
40     }
41   }
42   // Validate the response
43   const responseValidation = validateResponse(parsedResponse);
44   if (!responseValidation.success) {
45     res.status(500).json({
46       error: 'Generated text is invalid'
47     });
48   }
49   return response;
50 }
```

The chat window on the right shows a conversation with an AI assistant. The user's prompt is: "What is the steps to 'Implement Playwright with TypeScript'? share the steps, after my permissions implement the changes." The assistant's response is: "GROQ CLIENT IMPLEMENTATION AND FIXES implementation file. Tell me if you want me to delete it. To run the server end-to-end, set groq_API_KEY in environment and start dev server (npm run dev). I can start the dev server and make a sample request if you want. Next steps (pick one): Start the dev server and make a sample request (requires groq_API_KEY), Run the project tests (npm test), Remove groqClientimpl.ts (cleanup), Add unit tests/mocks for the Groq client. Which should I do next? (If you want to run the server, please confirm groq_API_KEY is set or provide one—do not paste secrets here; I can proceed only if the environment has it.)"

The chat window also shows a message from the user: "What is the steps to 'Implement Playwright with TypeScript'? share the steps, after my permissions implement the changes." and a response from the assistant: "Routed to gpt-5-mini. Asking which level of implementation to perform next so changes can be applied correctly — choose Full to add packages, configs, example tests, scripts, and CI workflow; choose Minimal to add only packages and configs; or choose to keep this as a guidance-only response. Question: Which implementation level do you want me to apply now? Answered: Full implementation (install deps, add playwright.config.ts, tsconfig updates, example tests, and CI workflow). Great — I'll implement the full Playwright + TypeScript setup: install dev dependencies, add TypeScript and Playwright config, create example tests, add npm scripts, and add a CI workflow. I'll proceed unless you want to adjust anything before I start."



New-Item -Path ".github\workflows" -ItemType Directory -Force

cd backend ; npm i --save-dev @playwright/test@^1.44.0 playwright@^1.44.0 --no-audit --no-fund

EXPLORER

USER-STORY-TO-TESTS-UPDATED-UI

.github

.vscode

backend

src

llm

routes

TS generate.ts

TS prompts.ts

TS schemas.ts

TS server.ts

tests

package.json

playwright.config.ts

tsconfig.json

tsconfig.playwright.json

frontend

node_modules

.env

.gitignore

package-lock.json

package.json

backend > src > routes > TS generate.ts > ...

8 generateRouter.post('/', async
15 error: 'Validation er
16 })
17 return
18 }
19 }
20 }
21
22 // Build prompts
23 const userPrompt = buildPrompt
24
25 // Create GroqClient instance
26 const groqClient = new GroqClient
27
28 // Generate tests using Groq
29 try {
30 const groqResponse = await
31
32 // Parse the JSON content
33 let parsedResponse: Generate
34 try {
35 parsedResponse = JSON.parse
36 } catch (parseError) {
37 res.status(502).json({
38 error: 'LLM returned invalid
39 }
40 return
41 }
42
43 // Validate the response
44 const responseValidation =
45 if (!responseValidation.succ

... powershell + - - - - -

PS F:\TestLeaf Training\Documents\GenAI
to-tests-updated-ui>

CHAT

GROQ CLIENT IMPLEMENTATION AND FIXES

4/4

Sure
changes enable running Playwright tests locally and in CI.
Proceeding to update package.json, create config and test files, and add CI workflow.
Reviewed 3 files
Updating backend package.json, adding Playwright config and TypeScript test tsconfig, creating
an example Playwright test, adding a GitHub Actions workflow, and installing Playwright and
browsers. Running installs so local dev and CI can run the tests. I'll modify package.json and
create the config/test/workflow files, then install dev dependencies and Playwright browsers.
Updated 3 files and created 3 test files +36 -2
Create tests directory under backend New-Item -Path "backend/tests" -ItemType Directory...
Updated example.spec.ts +7 -0
Create .github/workflows directory New-Item -Path ".github/workflows" -ItemType Direc...
Finished with 1 step +30 -0
Install Playwright dev dependencies cd backend ; npm i --save-dev @playwright/test@1... >
Install Playwright browsers cd backend ; npx playwright install --with-deps >
cd backend ; npx playwright install --with-deps
| 30% of 191.8 MiB
| 40% of 191.8 MiB
| 50% of 191.8 MiB
| 60% of 191.8 MiB
| 70% of 191.8 MiB
+ TS generate.ts
Chat with Copilot
+ Agent Auto
Copilot Interactive Default permissions

Type here to search

23°C

01:19

20-08-2026

All Playwright setup completed.
Summary of changes made

package.json (backend)

Added scripts: test:e2e, test:e2e:headed, test:e2e:report
Added devDependencies: @playwright/test and playwright

Added files (backend)

playwright.config.ts
tsconfig.playwright.json
tests/example.spec.ts

.github/workflows/playwright.yml — runs Playwright tests on push and PRs to main/master

Installed packages and browsers

Ran npm install for Playwright packages
Ran npx playwright install --with-deps to download browsers

How to run locally

From backend folder:

npm ci (to install deps)

npx playwright test (run tests headless)

npx playwright test --headed (run with browser visible)

npx playwright show-report (view HTML report after tests)

Notes and recommendations

Tests hit external site example.com; for E2E against local server, add a test which starts the dev server (or use playwright's fixtures to spawn the server) and point tests to http://localhost:PORT.

Playwright installed browsers are fairly large — CI will download them during workflow steps.

If you want tests to run as part of npm test, add a wrapper script or update existing test script.

Would you like:

An E2E test that starts the local dev server before running tests (and tears it down after)?

Cleanup by removing groqClientImpl.ts (left from earlier)?

Add a script to run Playwright tests as part of npm test?

Ask Mode — analysis of impacted files and plan

Summary of analysis

Goal: Implement Playwright + TypeScript E2E that runs against the project and is CI-ready.

Existing project state (relevant files found under backend):

backend/package.json — scripts and devDependencies (already updated earlier to include Playwright libs).

backend/tsconfig.json — existing TypeScript config for the app.

backend/tsconfig.playwright.json — created earlier to scope Playwright types.

backend/playwright.config.ts — created earlier but currently only includes testDir, projects, reporters, etc.

backend/tests/example.spec.ts — created earlier and currently hits example.com (external).

backend/src/server.ts — app server (listens on PORT or 8080) and provides a health endpoint /api/health.

backend/src/... (routes, etc.) — server endpoints the tests should exercise (/api/health, /api/generate-tests).

.github/workflows/playwright.yml — CI workflow created earlier that installs deps and runs Playwright tests.

Files that will be impacted/added by a complete Playwright+TypeScript implementation

backend/package.json

Add/start scripts to run the server in a way CI and Playwright webServer can use (e.g., "start:server": "tsx src/server.ts" or "start:server": "node dist/server.js" depending on preference).

Ensure scripts to run Playwright tests are present (test:e2e etc.). (Already added earlier, but will be updated if needed.)

backend/playwright.config.ts

Add webServer configuration so Playwright starts the backend automatically before tests and tears it down afterward.

Set baseUrl for tests to point to the server.

Ensure tsconfig usage or reference to tsconfig.playwright if needed.

backend/tsconfig.playwright.json

Ensure Playwright types are included and tests compile with TypeScript.

backend/tests/*

Replace example.com test with tests that target the local server (e.g., health endpoint, and an example API call to /api/generate-tests with a mocked/stubbed LLM if desired).

Add a test that uses baseUrl and checks /api/health and potentially an integration call to generate endpoint (for the latter, using deterministic behavior or mocking LLM responses may be required).

.github/workflows/playwright.yml

Ensure CI runs npm ci in backend, installs Playwright browsers, and then runs tests. Confirm Playwright's webServer command works in CI (i.e., that tsx is available or use "node dist/server.js" after build).

Optional/Helpful files

backend/.env.ci or documentation instructing env vars needed (groq_API_KEY not required for a simple health check; if E2E covers /api/generate-tests you must ensure environment either provides groq_API_KEY or mock the Groq client).

Add a Playwright fixture or configuration to provide env vars to the test run if needed.

Optionally add a helper to start server in debug/headed runs (script for running server in foreground).

Risks and decisions to make before implementing

How to start the server during tests:

Use Playwright's webServer option with a command that runs the server directly (recommended).

That requires the command to be available in CI (tsx or node).

Two options:

"npx tsx src/server.ts" — runs TypeScript directly (requires tsx installed as devDependency and npm ci in CI).

"node dist/server.js" — requires a build step in CI prior to running tests (npm run build).

Recommendation: use "npx tsx src/server.ts" as the webServer command and add a convenience script "start:server": "tsx src/server.ts" to package.json. This avoids build step in CI and is quick for local development. (tsx is already a devDependency.)

Tests that call /api/generate-tests:

The generate route calls the Groq client which expects an API key and may attempt outwards calls.

For robust E2E without secrets, tests should:

Limit to lightweight checks (health endpoint), or

Mock the Groq client (or set env vars and use a test-specific stub) — more work.

Recommendation: Start with health-check E2E and optionally add a test for the generate route that stubs/mocks the Groq client or supplies a fake groq response. Implementing mocking will require code changes to allow DI of a mock client or using environment variable to switch to a "mock" mode.

CI considerations:

Playwright browsers are large and CI step already uses npx playwright install --with-deps; keep that.

Ensure test timeout and webServer timeout are sufficient.

Plan to implement (Agent Mode)

Update backend/package.json:

Add a "start:server" script: "tsx src/server.ts"

Ensure test scripts exist (they were added earlier).

Update backend/playwright.config.ts:

Add webServer config:

command: "npm run start:server"

port: 8080 (or use process.env.PORT)

timeout: e.g., 30_000 or 60_000

reuseExistingServer: false (CI: always start)

Set use.baseURL to "http://localhost:8080" to make tests easier.

Update backend/tests/example.spec.ts:

Replace example.com check with a health endpoint check:

await page.goto('/') or use request fixture to GET '/api/health' with baseURL and assert status + JSON structure

Use playwright test API with request or page.

Update .github/workflows/playwright.yml

Ensure it runs from backend and that playwright will start server via webServer. (Already runs npm ci and playwright install.)

Add optional note or file for mocking later.

Run a local Playwright test (npx playwright test) to verify (I will run tests here).

If that plan looks good, I will switch to Agent Mode and implement these changes now.

Which choice:

Confirm to proceed with the implementation plan above (recommended).

Or request changes (e.g., prefer node-dist start, or add generate-route test with mocking now).

Please confirm which start-server command to use in webServer:

Use "npm run start:server" which runs "tsx src/server.ts" (recommended)

Or use "node dist/server.js" (requires building first in CI)

(Confirming this lets me implement webServer and tests without back-and-forth.)